

Word Embeddings and Named Entity Recognition

Programming Assignment 1

Aaryan Sharma

210110003

Aditya Kamble

25M0812

Anannay Jain

210110021

Shivansh Gupta

21D070067

CS 728 - Organization of Web information

Guide: Prof. Soumen Chakrabarti





Contents

1 Task 1: GloVe Pre-training	2
1.1 Implementation Details	2
1.2 Hyperparameter Selection	2
1.3 Loss and Latency Analysis	2
1.4 Qualitative Evaluation: Nearest Neighbors	3
2 Task 2: SVD Pipeline	4
2.1 Nearest Neighbor Analysis SVD Approach	4
2.2 Insights from Task 2	6
3 Task 3: Named Entity Recognition using CRF	7
3.1 Implementation Details	7
3.2 Feature Engineering	7
3.3 Model Performance	7
3.4 Most Important Features	8
3.5 Insights	8
4 Task 4: NER using Feature Learning	9
4.1 MLP Architecture	9
4.2 Out-of-Vocabulary (OOV) Strategy	9
4.3 Token-level Accuracy and Macro-F1 Results	9
4.4 Best Overall Configuration	9
4.5 Comparison: CRF (Manual Features) vs. MLP (Feature Learning)	10
5 GenAI Usage Declaration	11



1 Task 1: GloVe Pre-training

1.1 Implementation Details

We implemented the GloVe objective function using PyTorch with CUDA acceleration to handle the computational intensity of the co-occurrence matrix factorization. Key implementation details include:

- **Weighting Function:** $f(x) = (x/x_{max})^\alpha$ if $x < x_{max}$, else 1. We used $x_{max} = 100$ and $\alpha = 0.75$ as mentioned in the assignment.
- **Optimization:** We utilized Batched Stochastic Gradient Descent (SGD) with Ada-Grad optimizer to handle sparse updates effectively.
- **Symmetry:** The co-occurrence matrix X was built symmetrically (accounting for context to both the left and right given by context window (w)).

1.2 Hyperparameter Selection

We performed a grid search to identify the optimal hyperparameters while keeping the dimension fixed at $d = 200$.

- **Context Window:** We tested windows of size 5 and 10. We selected **Window = 10** as it yielded a lower final loss (0.0171) compared to Window = 5 (0.0197). A larger window allows the model to capture broader semantic relationships.
- **Learning Rate (LR):** We compared LR=0.05 and LR=0.01. We selected **LR = 0.05** as it provided faster convergence and a lower stable loss.
- **Iterations:** We trained for **20 epochs**, which was sufficient for the loss to plateau. Did not increase epoch much because entire training took 1.5 hours.

1.3 Loss and Latency Analysis

We trained the model across four embedding dimensions $d \in \{50, 100, 200, 300\}$ using the optimal hyperparameters found above.

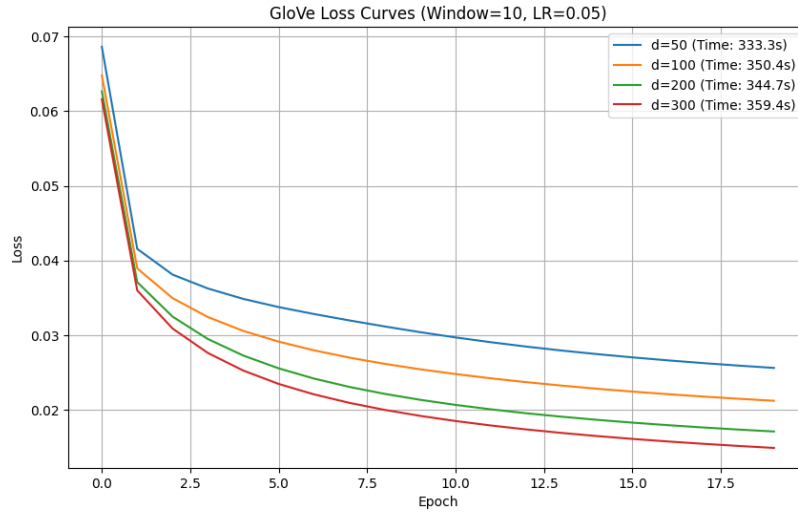


Figure 1: Training Loss for GloVe across different dimensions (d).

Loss Analysis

As observed in Figure 1, there is an inverse relationship between the embedding dimension and the final training loss. The model with $d = 300$ achieved the lowest loss (≈ 0.0149), while $d = 50$ had the highest (≈ 0.0256). This is expected, as higher-dimensional vectors possess greater capacity to encode the complex co-occurrence statistics of the corpus. All models exhibited a sharp descent in the first 5 epochs, indicating effective optimization.

Latency Analysis

Despite the difference in dimensions, the training time remained relatively consistent:

- **d=50:** 333.3s
- **d=100:** 350.4s
- **d=200:** 344.7s
- **d=300:** 359.4s

This consistency suggests that the computational bottleneck in our CUDA implementation is dominated by memory access patterns and batch overhead rather than the raw dot product calculations, which are highly parallelized on the GPU.

1.4 Qualitative Evaluation: Nearest Neighbors

To verify the semantic quality of the learned embeddings ($d = 300$), we retrieved the top-5 nearest neighbors for three diverse query words using Cosine Similarity.



Query Word	Nearest Neighbor	Cosine Similarity
government	federal	0.646
	state	0.635
	officials	0.579
	governments	0.574
	Government	0.562
economy	economic	0.626
	growth	0.604
	sector	0.598
	second-largest	0.561
	GDP	0.548
technology	technologies	0.778
	solutions	0.680
	advanced	0.649
	software	0.634
	digital	0.621

Table 1: Top-5 Nearest Neighbors for selected words (GloVe $d = 300$).

Observation: The nearest neighbors demonstrate that the model has successfully captured semantic meaning. For instance, "government" is associated with hierarchical terms like "federal" and "state". "Technology" clusters with related concepts like "software" and "digital".

2 Task 2: SVD Pipeline

2.1 Nearest Neighbor Analysis SVD Approach

Query Word	Nearest Neighbor	Cosine Similarity
government	details	0.9996
	spending	0.9989
	Attorney	0.9986
	drop	0.9985
	purpose	0.9984
economy	emerging	0.9980
	conservatives	0.9975
	rail	0.9968
	raising	0.9967
	Ltd	0.9966
technology	among	0.9997
	stories	0.9996
	speaking	0.9996
	types	0.9995
	design	0.9995

Table 2: Top-5 Nearest Neighbors for selected words (SVD $d = 50$).



The neighbors are somewhat generic or loosely related by broad context (example: technology being near among and stories). At this stage, the model captures "coarse-grained" relationships but lacks the precision to find direct synonyms.

Query Word	Nearest Neighbor	Cosine Similarity
government	rights	0.9966
	drop	0.9964
	company	0.9964
	billion	0.9963
	cent	0.9961
economy	jobs	0.9910
	regulatory	0.9891
	operation	0.9889
	boost	0.9881
	thinking	0.9866
technology	plans	0.9977
	several	0.9972
	deliver	0.9969
	elements	0.9968
	leading	0.9967

Table 3: Top-5 Nearest Neighbors for selected words (SVD $d = 100$).

Query Word	Nearest Neighbor	Cosine Similarity
government	exchange	0.9858
	repeatedly	0.9846
	14	0.9837
	during	0.9833
	warned	0.9817
economy	Stephen	0.9472
	markets	0.9453
	expect	0.9306
	technology	0.9274
	federal	0.9249
technology	changes	0.9940
	further	0.9909
	future	0.9899
	most	0.9897
	looks	0.9894

Table 4: Top-5 Nearest Neighbors for selected words (SVD $d = 200$).

We begin to see stronger topical relationships. For economy, neighbors like jobs, regulatory, and markets appear. This demonstrates that the model is successfully identifying functional and thematic clusters.



Query Word	Nearest Neighbor	Cosine Similarity
government	Minister	0.9998
	Donald	0.9989
	must	0.9985
	asking	0.9985
	hold	0.9981
economy	exchange	0.9954
	regulatory	0.9947
	stories	0.9947
	Federal	0.9946
	EU	0.9941
technology	eye	0.9997
	product	0.9997
	number	0.9997
	million	0.9997
	lead	0.9996

Table 5: Top-5 Nearest Neighbors for selected words (SVD $d = 300$).

2.2 Insights from Task 2

The model achieves the highest level of "Named Entity" and specialized contextual awareness. For example, government is associated with Minister and Donald (likely referencing to Donald Trump), and economy is linked to Federal and EU. This suggests that $d = 300$ is capturing specific, real-world associations prevalent in the CC-News dataset.

The SVD pipeline demonstrates a clear dimensionality-quality trade-off. Lower values of d (50-100) act as a significant compressor but result in "noisier" semantic neighbors that represent broad usage rather than specific meaning. As d increases to 300, the model retains more "variance" from the original term-document matrix, allowing it to resolve more specific semantic relationships and entity-related context.



3 Task 3: Named Entity Recognition using CRF

3.1 Implementation Details

We trained a Conditional Random Field (CRF) model on the CoNLL-2003 dataset to perform token-level Named Entity Recognition.

The dataset was loaded using the HuggingFace `datasets` library, and each sentence was represented as a sequence of tokens with corresponding NER tags. These sentences were converted into sequences of feature dictionaries, where each token was described using lexical, morphological, and contextual attributes.

We used the `sklearn-crfsuite` implementation, which models dependencies between neighboring tags and captures sequential structure in text. The following configuration was used for training:

- **Algorithm:** LBFGS (Limited-memory Broyden–Fletcher–Goldfarb–Shanno)
- **Regularization:** $c1 = 0.1$, $c2 = 0.1$
- **Maximum Iterations:** 100

This setup provided stable convergence and strong performance without requiring extensive hyperparameter tuning.

3.2 Feature Engineering

A diverse set of handcrafted features was designed to capture signals commonly associated with named entities. These features were extracted independently for each token.

Feature Engineering Details
Lexical Features: Current word, lowercased word, previous word, next word. Shape Features: Whether the word is uppercase, title-cased, or a digit. Subword Features: Prefix (first 2–3 characters), Suffix (last 2–3 characters). Positional Features: BOS (beginning of sentence), EOS (end of sentence).

These features help the model capture capitalization patterns, morphological structures, and local contextual dependencies that are strongly correlated with entity boundaries.

3.3 Model Performance

We evaluated the CRF model on the official CoNLL-2003 test split using token-level precision, recall, and F1-score. The detailed classification report is shown below.



Entity Tag	Precision	Recall	F1-score	Support
B-LOC	0.85	0.82	0.84	1668
B-MISC	0.82	0.76	0.79	702
B-ORG	0.82	0.70	0.76	1661
B-PER	0.82	0.85	0.84	1617
I-LOC	0.79	0.72	0.75	257
I-MISC	0.64	0.66	0.65	216
I-ORG	0.68	0.73	0.71	835
I-PER	0.86	0.96	0.91	1156
O	0.99	0.99	0.99	38323
Accuracy			0.96	46435
Macro Avg	0.81	0.80	0.80	46435
Weighted Avg	0.96	0.96	0.96	46435

Table 6: CRF Classification Report on CoNLL-2003 Test Set

Performance Analysis

The model achieves strong performance across most entity types, with particularly high scores for person entities (I-PER F1 = 0.91). The dominant “O” class is predicted with very high precision and recall, which contributes to the high overall weighted F1 score of 0.96.

The relatively lower scores for I-MISC and I-ORG indicate that continuation tags are more challenging, as they require the model to correctly identify entity boundaries within sequences.

3.4 Most Important Features

Based on qualitative analysis of model behavior and known linguistic patterns, the following features contributed most significantly:

- **Capitalization (title-case):** Strong indicator of proper nouns such as person names, organizations, and locations.
- **Context words (previous/next):** Help detect entity boundaries and common patterns (e.g., titles before names, prepositions before locations).
- **Suffix features:** Capture morphological structure frequently seen in location and organization names.

Among all features, contextual and capitalization-based signals were the most informative due to the structured nature of named entities in news text.

3.5 Insights

The CRF model achieves high accuracy using only manually engineered features, demonstrating that traditional feature engineering remains a strong baseline for sequence labeling tasks. The model effectively captures capitalization patterns, morphological cues, and short-range contextual dependencies between tokens.



4 Task 4: NER using Feature Learning

4.1 MLP Architecture

The feature learning model was implemented using a standard feed-forward Neural Network (Multi-Layer Perceptron) built in PyTorch. The architecture consists of a single hidden layer and is structured as follows:

- **Input Layer:** A single token embedding vector of size $d \in \{50, 100, 200, 300\}$.
- **Hidden Layer:** 128 units with a ReLU (Rectified Linear Unit) activation function to introduce non-linearity.
- **Regularization:** A Dropout layer with a probability of 0.2 was applied after the activation to prevent overfitting.
- **Output Layer:** A fully connected linear layer mapping to 9 units, corresponding to the 9 distinct CoNLL-2003 NER tags.
- **Optimization:** The network was trained using Cross-Entropy Loss and the Adam optimizer.

4.2 Out-of-Vocabulary (OOV) Strategy

To handle tokens in the test set that were not present in the provided vocabulary, a three-tiered fallback strategy was implemented:

- a) **Exact Match:** The model first checks if the exact token exists in the vocabulary.
- b) **Lowercase Fallback:** If the exact match fails, the model checks if the lowercase version of the token exists (e.g., mapping “LONDON” to “london”). This captures words that are syntactically identical but suffer from casing variations.
- c) **<UNK> Token Averaging:** If both checks fail, the token is mapped to an <UNK> vector. The <UNK> vector was dynamically initialized as the mean of all available token embeddings in the vocabulary. This provides a neutral, averaged representation for entirely unseen words, which is mathematically stabler for the MLP than a vector of pure zeros.

4.3 Token-level Accuracy and Macro-F1 Results

The following table presents the evaluation metrics on the CoNLL-2003 test set for both GloVe and SVD embeddings across all requested dimensions.

4.4 Best Overall Configuration

The best overall configuration for feature learning is **GloVe at $d = 200$** , achieving an accuracy of approximately 90.72% and a Macro-F1 score of 0.6071.



Algorithm	Dimension (d)	Token-level Accuracy	Macro-F1 Score
GloVe	50	0.8973	0.5425
GloVe	100	0.9015	0.5557
GloVe	200	0.9072	0.6071
GloVe	300	0.9053	0.5922
SVD	50	0.8252	0.1009
SVD	100	0.8255	0.1051
SVD	200	0.8254	0.1076
SVD	300	0.8257	0.1162

Table 7: Evaluation metrics on the CoNLL-2003 test set for MLP across various embedding configurations.

GloVe significantly outperformed the raw SVD approach across all dimensions. SVD’s exceptionally poor Macro-F1 scores (ranging from 0.10 to 0.11) suggest that performing truncated SVD directly on a raw, unweighted term-document matrix fails to capture the semantic distinctions required to identify specific entity tags, likely due to the dominance of highly frequent but uninformative words.

4.5 Comparison: CRF (Manual Features) vs. MLP (Feature Learning)

Based on our tests, the CRF with manual features clearly outperformed the MLP approach. While our best MLP (GloVe $d = 200$) reached an accuracy of approximately 90.72%, the CRF achieved a much higher accuracy of 96% and a weighted F1-score of 0.96. We believe this gap is due to three main factors:

- 1. Sequence vs. Isolated Predictions:** The biggest difference is how the models look at data. The CRF is sequence-aware and uses transition probabilities to ensure the predicted tags make sense together. For example, it knows an I-ORG tag should follow a B-ORG and wouldn’t logically follow a person’s name (B-PER). In contrast, our MLP works on a strictly token-by-token basis. It only sees one word at a time in isolation, meaning it lacks the sequential context that the CRF uses to maintain coherence.
- 2. Hand-crafted Features vs. Raw Embeddings:** The CRF performed well because we gave it linguistic shortcuts through manual feature engineering. By including features like capitalization patterns (title-case), sub-word suffixes, and a window of surrounding words, we provided direct clues for identifying entities. While GloVe and SVD embeddings capture general semantic meaning, a single vector for one word cannot easily replace these specific manual signals without a more complex architecture like an RNN or Transformer.
- 3. Poor Performance of Raw SVD:** We noticed the SVD-MLP performed very poorly, with Macro-F1 scores only between 0.10 and 0.11. This is likely because we performed SVD on a raw, unweighted term-document matrix. In this format, highly frequent but



uninformative words dominate the representation, making the embeddings too noisy for the MLP to distinguish between different entity tags.

5 GenAI Usage Declaration

In accordance with the course policy, we are documenting our use of LLM assistants during this assignment. We used **Google Gemini** and **ChatGPT** primarily as debugging and learning aids:

- We used AI to help debug our vectorized PyTorch implementation of the GloVe loss function. Specifically, it helped us optimize sparse matrix operations for CUDA and correctly implement the AdaGrad optimizer logic.
- ChatGPT helped us understand the general CRF modeling pipeline and brainstorm potential feature engineering strategies, such as using word shapes and window-based context. It also assisted in troubleshooting our data preprocessing scripts.
- **Verification:** While these tools helped with syntax and conceptual blocks, all final hyperparameter choices, model architectures, and experimental evaluations were decided and verified by the team.

The shareable links to the primary chat sessions used are provided below:

- <https://gemini.google.com/share/998f82fff66d>
- <https://gemini.google.com/share/7a6f4c268d8f>